Manual – Exercise 8: Homomorphic Encryption

How to run:

To run the solution, open a terminal in the corresponding directory, ensure the correct Python environment is active, and execute pytest to run all tests.
All tasks are validated through these tests, so no manual input or additional setup is required. Task4 might not work on windows or WSL.

Task 1 — ElGamal Cryptosystem (Multiplicative HE) (slide: 16)

What was implemented

- Standard ElGamal encryption over a multiplicative group modulo prime `p`
- Public key: `(p, g, y)`, Private key: `x`
- Homomorphic operation:

```
E(m1) ⊗ E(m2) = E(m1 · m2 mod p)
```

Example

Messages:

```
m1 = 7
m2 = 11
```

Server multiplies ciphertexts, client decrypts:

```
7 · 11 = 77 mod p
```

Difference to lecture theory

- We followed the slide definition (textbook ElGamal)
- Plaintexts are used directly as group elements (integers in Zp*)
- In real systems, **message encoding into the group** and **large primes** are required for security

(a) Tests

- `test_elgamal.py` verifies:
  - Basic functional correctness (check if encrypt and decrypt message are the same): test_encrypt_decrypt_basic, test_encrypt_decrypt_multiple_messages
  - Boundary and edge-case testing (explicitly test smallest an largest valid messages(1, p-1)): test_encrypt_decrypt_edge_values, test_invalid_message_range
  - Key-related behavior and randomness (test that the key is generated in some random manner): test_encrypt_with_new_key_each_time, test_ciphertext_randomness
  - Homomorphic properties and algebraic structure (test if fundamental multiplicative homorphism holds during encryping and decryping): test_homomorphic_multiplication, test_homomorphic_chained, test_homomorphic_identity
  - negative crypographic test (check if the message is really encrypted 2 x differently): test_wrong_key_fails

Task 2 — Paillier Shopping Cart (Additive HE) (slide: (wikipedia,))

What was implemented

- Privacy-preserving shopping cart using Paillier from the `phe` library
- Client encrypts **quantities**
- Server keeps **prices** in clear and computes **encrypted total**

Supported operations:

```
E(a) + E(b) = E(a + b)
E(a) · k    = E(a · k)
```

Example

Cart:

```
(2000, 1)   # 20.00
(120,  5)   # 1.20 each
→ total = 2000 + (120·5) = 2600
```

Server returns encrypted total, client decrypts → **2600**.

Privacy property

Server **never learns quantities**, only fixed prices.

Tests

- `test_paillier_cart.py` checks total against plaintext reference

(a) Which operations does the Pailler encryption scheme support?

The Paillier encryption scheme supports additive homomorphism, meaning encrypted values can be added together, and ciphertexts can be multiplied by a known plaintext scalar.

(b) Which data types does it operate on?

It operates on integers modulo n (specifically values in $\mathbb{Z}_n$), where n is part of the public key; in our implementation these are standard Python integers within that range

Task 3 — ElGamal Shopping Cart (Limitations)

What was implemented

- Same shopping cart model but using ElGamal (multiplicative-only HE)
- Subtotals are encrypted
- Server combines encrypted values **multiplicatively**

Result

```
Subtotal1 = 20
Subtotal2 = 15

Server computes: 20 · 15 = 300
Client decrypts: 300   (but correct sum is 35)
```

Conclusion

ElGamal **cannot** compute sums homomorphically.
This demonstrates why **Paillier is the appropriate scheme** for the shopping cart scenario.

Tests

- `test_elgamal_cart.py` shows product result differs from expected sum

(a) Which changes did you have to make, and why?

Compared to Task 1, we extended ElGamal from encrypting a single number to encrypting multiple shopping-cart subtotals, and we added client/server classes to mimic the outsourced-computation scenario from the Paillier task. Because ElGamal only supports multiplication, we had to change the server logic from computing a sum (like Paillier) to computing a product of encrypted values, which no longer corresponds to a meaningful cart total.

(b) What do your changes mean for the client (which advantages and/or disadvantages does the client have)?

For the client this means that, unlike in Task 2, the server cannot compute the total price privately on encrypted data — the client would have to compute the final sum itself. Thus, the client no longer benefits from outsourcing the sensitive computation, because ElGamal's multiplicative homomorphism does not support the required addition needed for real shopping-cart totals.

Task 4 — Fully Homomorphic Encryption (FHE Mean)

What was implemented

- Code using the `concrete` FHE compiler that computes:

```
mean = (x1 + ... + x6) // 6
mean_scaled = (sum * 100) // 6
```

- Designed according to lecture slides (see p. 5):
  encrypted computation without decryption at the server

4(a) How can you obtain the mean of a list with seven encrypted integers?

- To compute the mean of seven encrypted integers, a new FHE circuit must be compiled that accepts 7 inputs instead of 6. FHE circuits require a fixed input shape, so the 6-input version cannot be reused. The client then encrypts 7 values, the server evaluates the new circuit, and only the client decrypts the result.

4(b) What are the limitations of your implementations?

- Our implementation works only for exactly 6 values, so any other number of inputs requires recompilation. The mean uses integer arithmetic with truncation due to scaling by 100 and floor division. Additionally, the FHE library could not be installed on our Windows/WSL setup, so the solution is conceptual only.

4(c) Provide the rationale for your test case selection. Why these test cases and not others? Argue why your selected test cases provide good test coverage.

- We selected test cases covering constant values, exact-division means, fractional means, and mixed values to exercise all arithmetic behaviors of the scaled mean formula. These cases ensure correct handling of addition, scaling, and truncation. In this limited integer domain, they provide good overall coverage for the intended functionality.

Execution Notes (Environment Limitation)

We attempted to install the required concrete-numpy / concrete-compiler libraries in multiple environments to execute Task 4:

- Windows (conda, Python 3.9)
  concrete-python / concrete-numpy are not published for Windows, so pip could not find any compatible wheels.
- WSL Ubuntu 24.04 (Python 3.12 default)
  concrete-numpy requires Python < 3.12, so all versions were rejected by pip.
- WSL Ubuntu 24.04 (Python 3.11 virtual environment)
  pip attempted to install a matching concrete-compiler version but failed with:
  “No matching distribution found for concrete-compiler==0.24.0rc5”
  indicating that no binary package exists for this OS/Python combination.

Since the Concrete toolchain does not provide wheels for our available platforms and no source builds are supported by pip, Task 4 cannot be executed locally despite correctly implementing the code. Therefore, the provided solution is conceptually correct but not run due to upstream platform limitations.

Tests

- `test_fhe_mean.py` contains **conceptual** test cases (expected plaintext values)
- These tests reflect desired correctness but were not run due to environment constraints

Update

We managed to get 'concrete-numpy' running late Saturday on a machine running Ubuntu natively in combination with Python 3.10. The implementation including a set of tests can be found in the file 'Ex8_Fully_Homomorphic_Encryption.py'. It can be run using the command 'python3 Ex8_Fully_Homomorphic_Encryption.py'. However, given the late implementation, please use the main document as reference in case of inconsistencies.

Cryptosystems Summary

Scheme	Homomorphism	Works for Shopping Cart	Task
ElGamal	Multiplication only	✗	1, 3

Scheme	Homomorphism	Works for Shopping Cart	Task
Paillier	Addition + scalar multiplication	✓	2
FHE (Concrete)	Arbitrary computations	✓	4

Files Overview

File	Description
`elgamal.py`	ElGamal scheme (Task 1)
`paillier_cart.py`	Paillier cart (Task 2)
`elgamal_cart.py`	ElGamal cart variant (Task 3)
`fhe_mean.py`	FHE mean computation (Task 4, code only)
`test_*.py`	PyTest validation for tasks 1–3 and conceptual checks for task 4

All tests for Tasks 1–3 pass successfully using `pytest` in the `FDSEx08` conda environment.

Conclusion

This exercise demonstrates how the **choice of homomorphic encryption scheme** determines which computations can be outsourced securely:

- ElGamal => multiplication only => insufficient for computing totals
- Paillier => secure addition => correct encrypted shopping cart
- FHE => most flexible => average computation possible, but more complex libraries required